ANEXO TÉCNICO DE EVIDENCIAS DE EJECUCIÓN

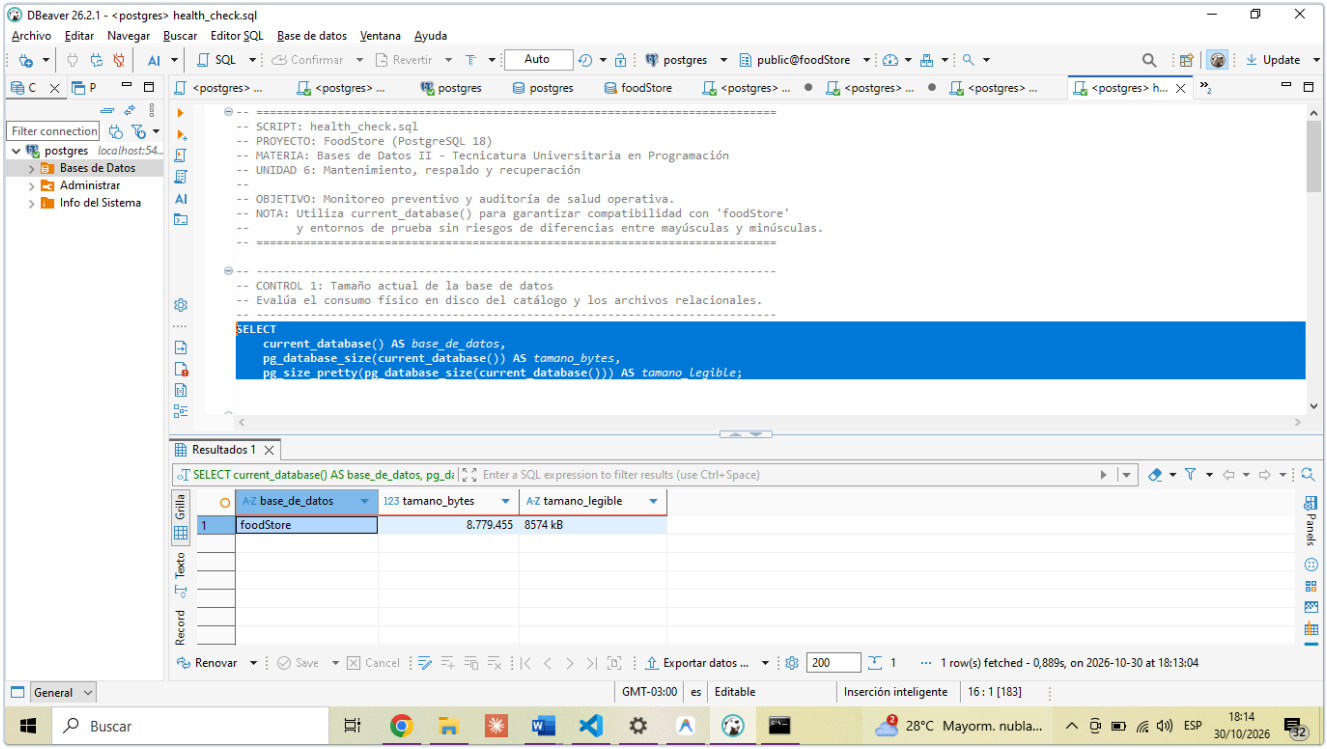
> **Materia:** Bases de Datos II (TUP) **Motor:** PostgreSQL 18 **Entorno:** Windows / PowerShell 5.1 / DBeaver
> **Base Principal:** `foodStore` **Base Descartable:** `foodstore_restore_test` **Fecha:** 30/10/2026

El presente anexo consolida las capturas de pantalla de la ejecución real en laboratorio de cada componente del Trabajo Práctico de la Unidad 6 (Monitoreo, Mantenimiento preventivo, Respaldo y Simulacro de recuperación ante desastres), acreditando los valores de telemetría y métricas reportados en el informe técnico.

Evidencia 1: Monitoreo Preventivo y Auditoría Operativa (health_check.sql)

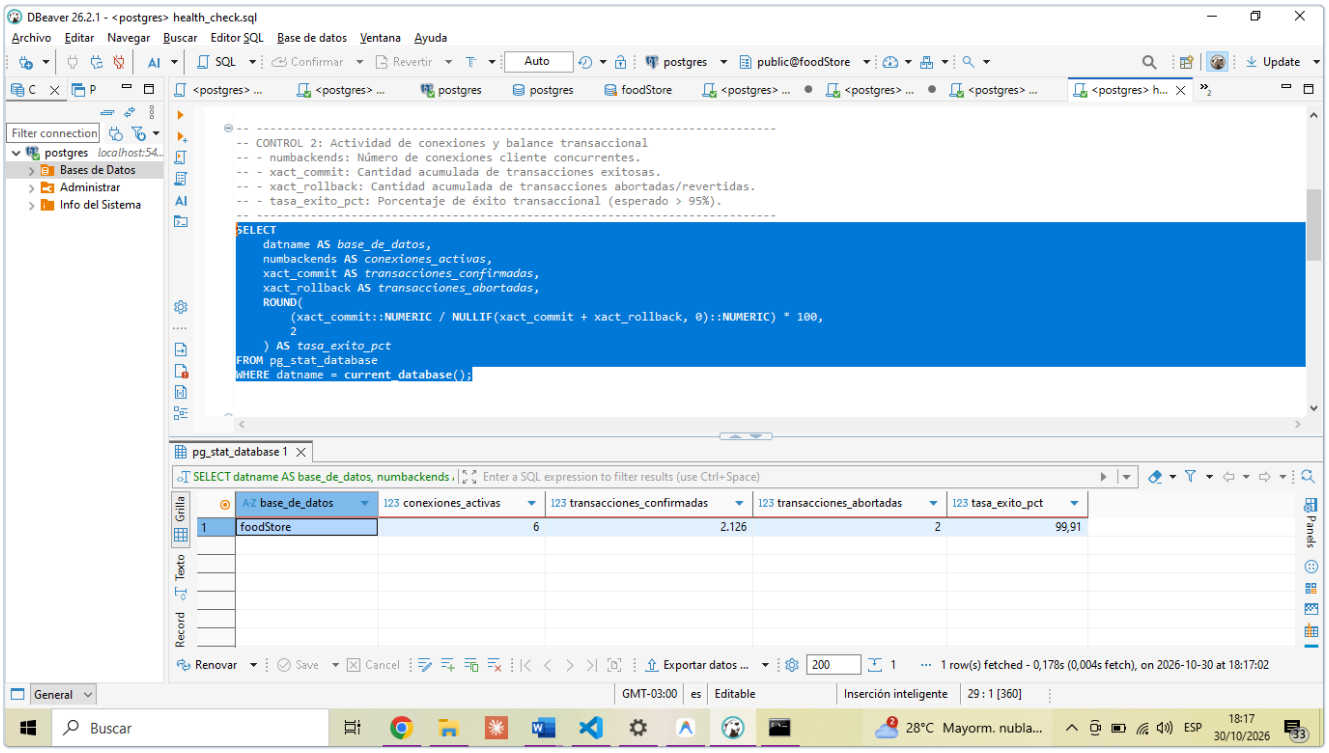
Control 1: Consumo físico en disco (Tamaño de Base)

Consulta ejecutada: `SELECT current_database(), pg_database_size(...), pg_size_pretty(...).`
Resultado observado: La base `foodStore` registra un tamaño físico en disco de **8.779.455 bytes (8.574 kB)**, estableciendo la línea base de capacidad.



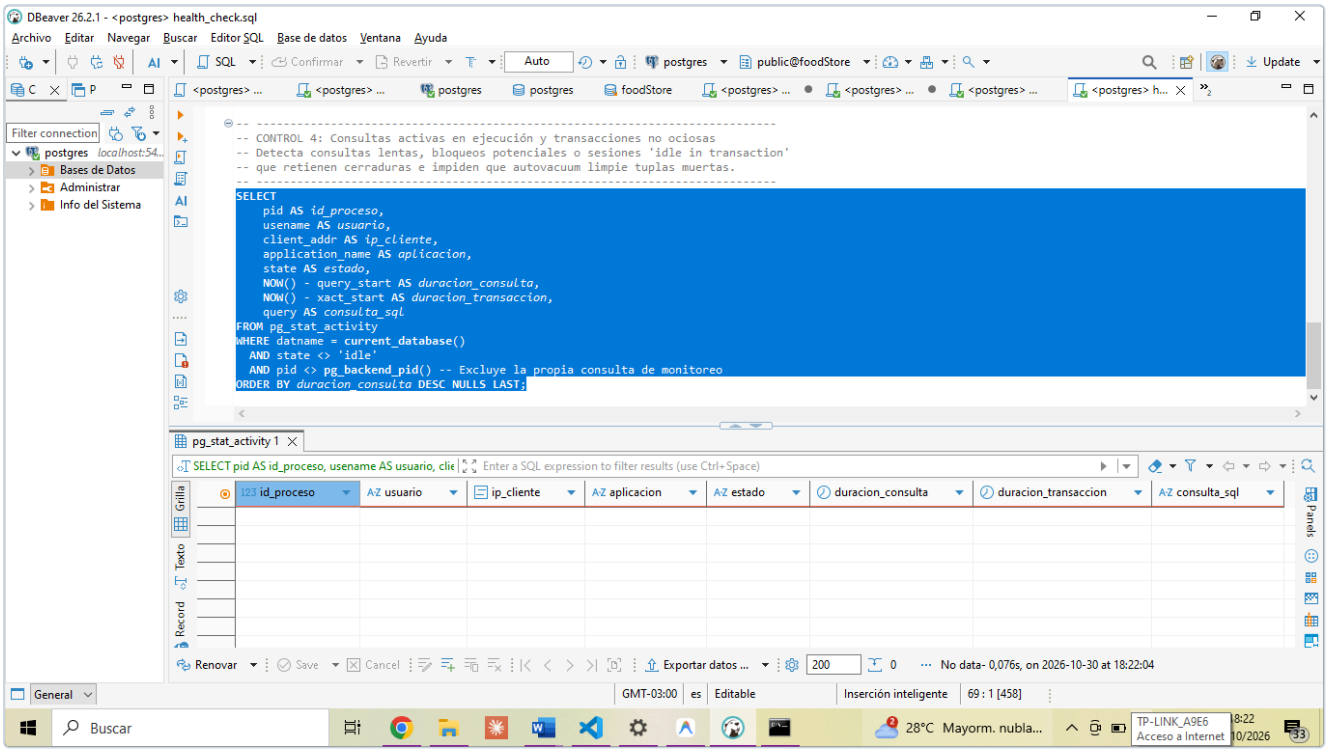
Control 2: Actividad de conexiones y balance transaccional

Consulta ejecutada: Monitoreo sobre `pg_stat_database` para la base activa.
Resultado observado: Conexiones concurrentes activas: **6**. Transacciones exitosas acumuladas: **2.126 commits** vs. solo **2 rollbacks**, arrojando una **tasa de éxito transaccional del 99,91%** (muy superior al umbral mínimo recomendado del 95%).



Control 4: Detección de consultas bloqueadas o colgadas

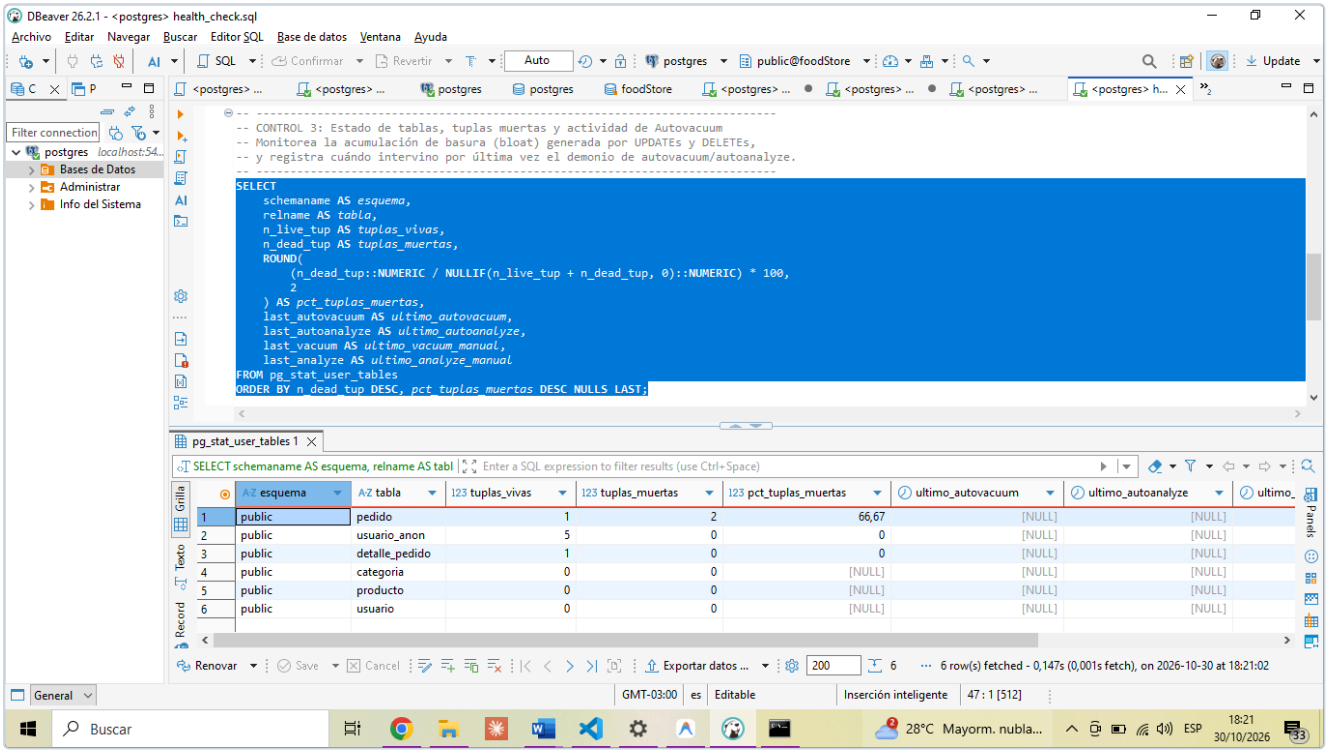
Consulta ejecutada: Filtrado en `pg_stat_activity` excluyendo estado `idle` y el proceso actual.
Resultado observado: 0 filas devueltas (No data). Confirma la ausencia total de bloqueos cruzados (deadlocks), transacciones abiertas ociosas (`idle in transaction`) o consultas lentas que obstaculicen la operación o impidan la limpieza del autovacuum.



Evidencia 2: Diagnóstico Previo de Tuplas Muertas

Control 3 / Diagnóstico Previo: Identificación de bloat en tablas

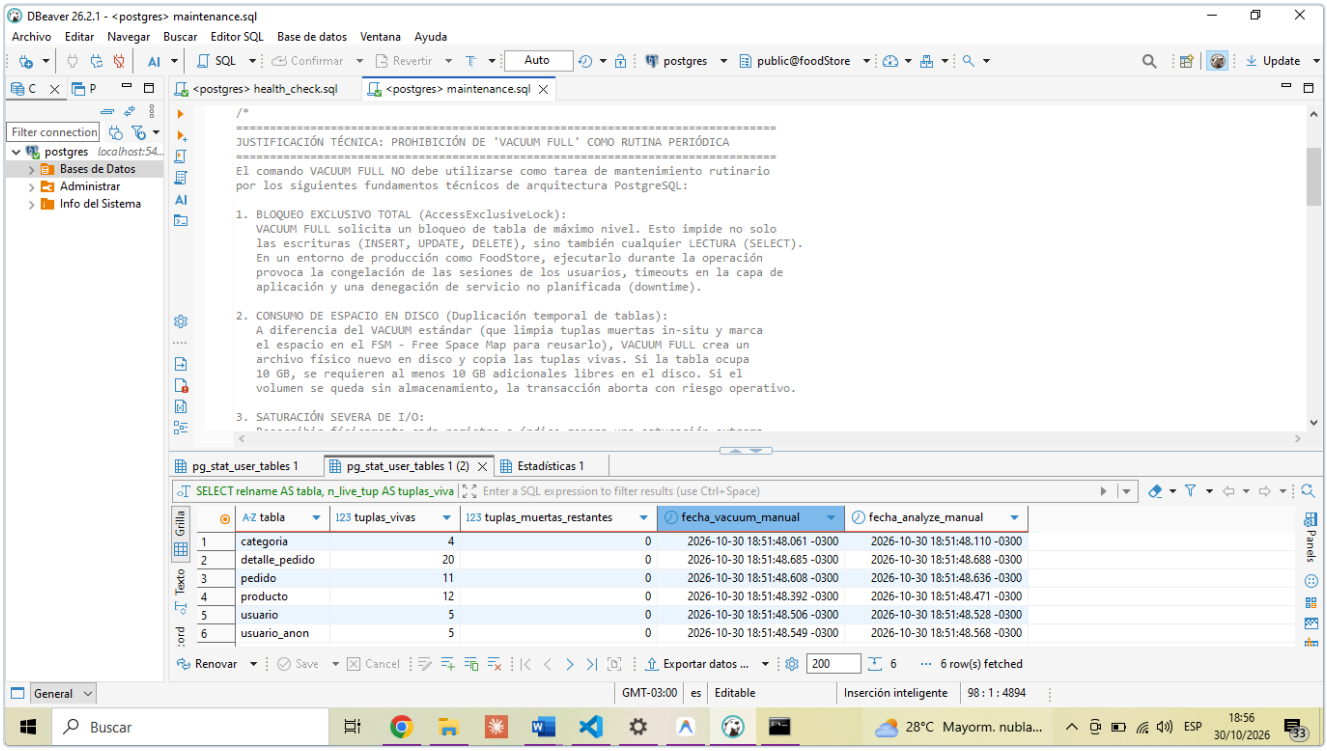
Consulta ejecutada: Inspección en `pg_stat_user_tables` ordenado por tuplas muertas descendente.
Resultado observado: Se detecta acumulación crítica de bloat en la tabla **`pedido`** con **2 tuplas muertas frente a 1 tupla viva (66,67% de tuplas muertas)**, generadas por operaciones de modificación/eliminación previas sin recolección de espacio. Este hallazgo **justifica técnicamente** la necesidad de ejecutar mantenimiento asistido.



Evidencia 3: Mantenimiento Seguro sin Bloqueo Exclusivo (maintenance.sql)

Verificación Post-Mantenimiento: VACUUM (VERBOSE, ANALYZE) concurrente

Acción aplicada: Ejecución de `VACUUM (VERBOSE, ANALYZE)` sobre `categoria`, `producto`, `usuario`, `usuario_anon`, `pedido` y `detalle_pedido`, evitando estrictamente `VACUUM FULL`.
Resultado observado: Reducción al 100% de tuplas muertas restantes (**`n_dead_tup = 0` en todas las tablas** del catálogo). Se certifica la actualización simultánea de las marcas temporales **`fecha_vacuum_manual` y `fecha_analyze_manual` a las 18:51:48**, garantizando que el optimizador de consultas cuente con estadísticas estadísticas actualizadas sin haber bloqueado lecturas ni escrituras en producción.



Evidencia 4: Simulacro de Desastre Contra Reloj (restore_test.ps1)

Ejecución automatizada en PowerShell y cronometraje de RTO/RPO

Procedimiento: Ejecución de `.\restore_test.ps1` en PowerShell 5.1 restaurando sobre `foodstore_restore_test` mediante `pg_restore --no-owner --no-privileges` a partir del dump `foodStore_2026-10-30_172759.dump`.
Métricas y Resultados Obtenidos:

- **RTO Medido: 6.18 segundos** (Meta: < 300 s / 5 min).
- **RPO Observado: 06h 00m 26s** (Meta: < 24 h).
- **Paridad en Conteo de Filas:** `categoria` = 4, `usuario` = 5, `producto` = 12, `pedido` = 11, `detalle_pedido` = 20 (100% coincidente).
- **Controles de Restricciones Lógicas:** `productos_invalidos = 0` y `detalles_invalidos = 0`.
- **Seguridad:** Contraseña ingresada de forma enmascarada (********) sin exposición en texto claro.

